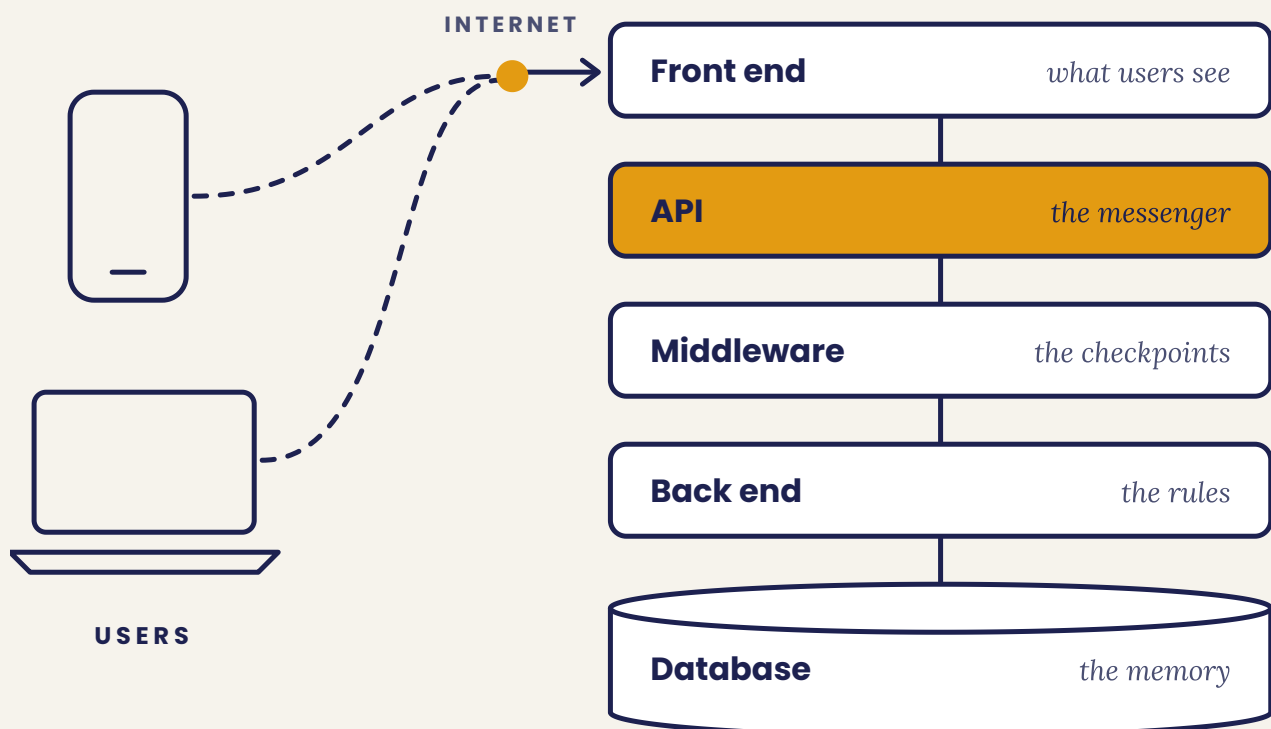


Fundamentals of Software Application Design

What every non-coder should know before using AI to build an App



AUTHOR

Kartik Vyas

Prepared for 2nd-year MICA students

About this guide

Audience: 2nd-year MICA students

Key assumption: No coding or computer science background. Every idea is explained in plain language, with everyday examples.

Goal: After reading this document, you should be able to explain the high-level architecture of any software app, whether it's Swiggy, Instagram, your bank's app or a college portal. That means being able to say:

- what runs where: on the user's phone or browser, or on servers
- which part talks to which part, and how
- where the data is stored
- how the app connects to the internet, to outside services and to other users

Why this matters for using LLMs and Gen AI to develop applications: An AI agent can write code for you, but you still decide what to build and how the pieces fit together. If you don't know the pieces, the agent decides for you, and you can't spot when it decides badly. Knowing the fundamentals lets you:

- give the agent clear, complete instructions
- recognise when it has put something in the wrong place (for example, a secret password inside the part of the app anyone can see)
- choose sensible tools instead of whatever the agent picks first
- fix problems by describing them in the right words

Note: The entire guide has been created with the aid of Claude and Claude Design.

How this guide is organised

Chapter	Topic	The question it answers
1	What software architecture is	What does "architecture" mean for an app?
2	Client-server architecture	How does my phone get information from somewhere else?
3	The three layers of an application	What are the main parts of every app?
4	Front end	What does the user see and touch?
5	Back end	Where do the rules and the real work happen?
6	Middleware	What sits in between and checks, translates or connects?
7	APIs	How do different parts talk to each other?
8	Databases and choosing one	Where is information stored, and which kind should I use?
9	Choosing a front-end framework	Which toolkit should the screens be built with?
10	Architecture styles	Should the app be one big piece or many small ones?
11	Deployment	How does an app get from a laptop onto the internet?
12	Security, scale, reliability and cost	What makes an app safe, fast and affordable?
13	Putting it all together	How do I read the architecture of any app?
—	Glossary	Quick definitions: foundational terms, then advanced ones

Running example used throughout: a food-delivery app, like Swiggy or Zomato. Everyone has used one, and it contains almost every idea in this guide.

CHAPTER 1 — What Is Software Architecture?

1.1 An app is a system, not a screen

When you open a food-delivery app, you see a few screens. But behind those screens are:

- a place where every restaurant, dish, price and order is stored
- rules such as "only show restaurants that are open and deliver to this address"
- connections to other companies for payments, maps and SMS messages
- computers somewhere in the world that stay switched on 24 hours a day

The screen is the tip of the iceberg. **Architecture** is the plan for the whole iceberg.

1.2 Definition

Software architecture is the high-level plan of an application. It shows:

1. **the main parts** (components) of the app
2. **what each part is responsible for**
3. **how the parts communicate** with each other
4. **where each part runs** (on your phone, on a company's computer, in the cloud)

Analogy: building a house. Before anyone lays a brick, an architect draws where the kitchen, bedrooms, plumbing and electricity go. You can repaint a room later easily. Moving the kitchen after the house is built is very expensive. Software is the same: good early decisions are cheap, and bad ones are costly to undo.

1.3 What good architecture is trying to achieve

Every architecture decision balances a few qualities. Engineers call these **non-functional requirements**: not *what* the app does, but *how well* it does it.

Quality	Plain-English question	Food-delivery example
Performance	Is it fast?	Does the menu load in under two seconds?
Scalability	Can it handle growth?	Does it survive 10× more orders on New Year's Eve?
Security	Is data protected?	Can one customer see another customer's address?
Reliability / availability	Does it keep working?	Is it up at 2 am on a Sunday?
Maintainability	Is it easy to change?	Can we add a "tip the rider" feature in a week?
Cost	Can we afford to run it?	What is the monthly bill for servers?

You rarely get all six at maximum. More speed or more safety usually costs more money or effort. **Architecture is the art of choosing the right trade-offs for your situation.**

CHAPTER 2 — Client–Server Architecture

2.1 The core idea

Almost every app you use follows one pattern:

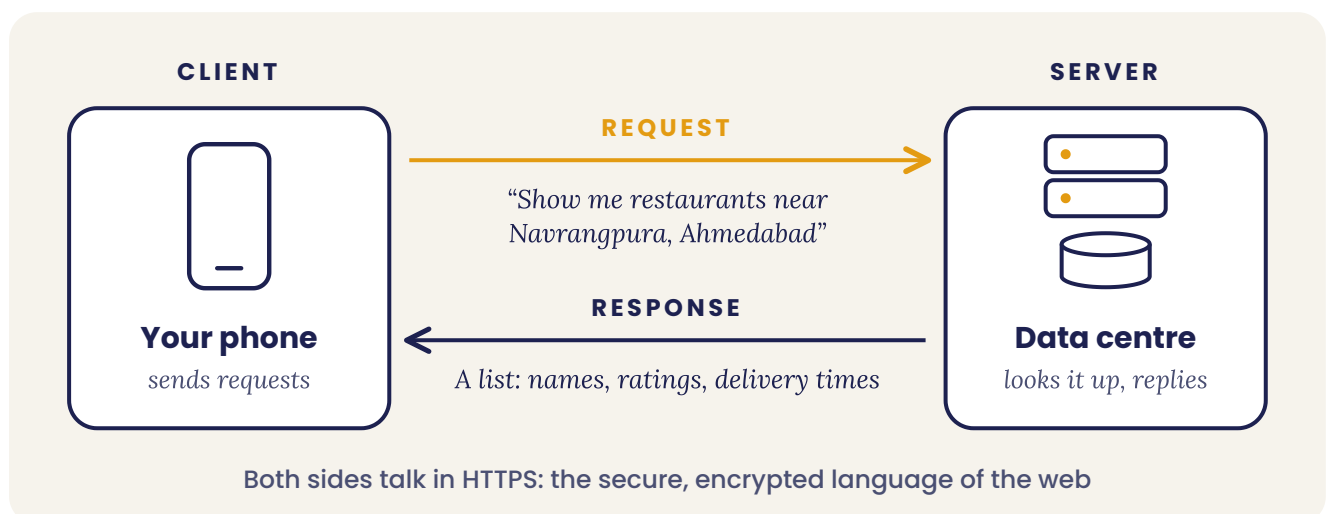
- A **client** asks for something.
- A **server** provides it.

	Client	Server
What it is	The device or program the user interacts with	A computer (or program) that waits for requests and responds
Examples	Your phone's Swiggy app, Chrome browser, a laptop	Computers in a data centre run by Swiggy or a cloud company
Job	Show things, collect input, send requests	Store data, apply rules, send back answers
How many	Millions	Relatively few, shared by everyone

Analogy: a restaurant. You (the client) sit at a table and place an order. The kitchen (the server) prepares it and sends it out. You don't go into the kitchen; you just make requests and receive results. One kitchen serves many tables.

2.2 Request and response

The conversation between client and server is always a **request** followed by a **response**.



The language they use to talk over the web is called **HTTP** (HyperText Transfer Protocol). **HTTPS** is the secure, encrypted version; the padlock in your browser means HTTPS is being used. Every modern app should use HTTPS.

2.3 What happens when you type a web address

This sequence is worth knowing because it contains the whole client-server story:

1. You type www.example.com into the browser (the client).
2. The browser asks the **DNS** (Domain Name System, the internet's phone book) for the numeric address of that name, called an **IP address**.
3. The browser sends an **HTTPS request** to the server at that IP address.
4. The server's back end **processes** the request and, if needed, reads from the **database**.
5. The server sends back a **response**: the page's content or data.
6. The browser **displays** the page to you.

2.4 Key words

- **URL**: a full web address, e.g. `https://www.example.com/menu`
- **Domain**: the name part, e.g. `example.com`, which you rent from a domain registrar
- **IP address**: the numeric address of a computer on the internet
- **DNS**: the service that turns domains into IP addresses
- **Stateless**: each request is handled on its own; the server doesn't automatically "remember" you between requests. That's why apps use **login tokens or cookies**, a kind of wristband that proves who you are on every request.

2.5 Other models (for completeness)

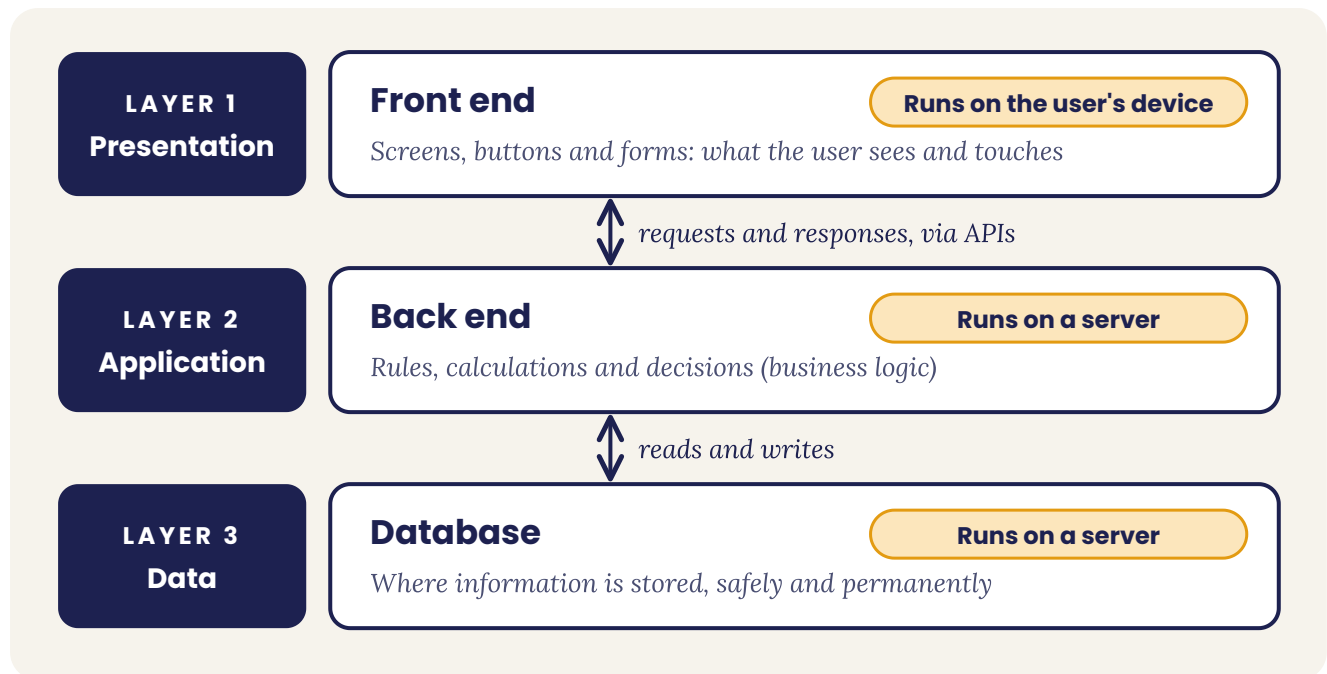
- **Peer-to-peer**: devices talk directly to each other without a central server, as in some file-sharing systems. Uncommon for business apps.
- **Offline / local apps**: everything runs on one device, like a calculator app. No server needed, but data can't be shared.

Most apps you will develop using LLMs and Gen AI are client-server apps.

CHAPTER 3 — The Three Layers of Every Application

3.1 The three-tier model

Nearly every application can be split into three layers:



3.2 The restaurant analogy, extended

Restaurant	Software	Food-delivery app
Dining room, menu card, table	Front end	Screens where you browse and order
Waiter who carries orders	API	The messages between app and server
Security at the door, order checker	Middleware	Checks you're logged in; logs every order
Kitchen and chefs	Back end	Calculates bill, assigns rider, applies coupon
Pantry and store room	Database	Stores users, restaurants, dishes, orders
Suppliers (vegetables, gas)	Third-party services	Payment gateway, maps, SMS

3.3 Why separate the layers?

- **Security:** the kitchen isn't open to the public, so sensitive logic and data stay hidden.
- **Flexibility:** you can redesign the dining room (front end) without rebuilding the kitchen.
- **Reuse:** one back end can serve a website, an Android app and an iPhone app at the same time.
- **Teamwork:** different people can work on different layers.

Lesson for using LLMs and Gen AI to develop applications: when you ask an agent for a feature, ask yourself which layer it belongs to. "Make the button blue" is front end. "Don't allow orders above ₹10,000 without verification" is back end. Putting a rule in the wrong layer is one of the most common and most dangerous mistakes.

CHAPTER 4 — The Front End

4.1 What it is

The **front end** (also called the **client side** or **UI**, user interface) is everything the user sees and interacts with: layouts, text, images, buttons, forms, animations.

4.2 The three building blocks of the web

Technology	Job	Analogy
HTML	The structure and content: headings, paragraphs, buttons, forms	The skeleton
CSS	The appearance: colours, fonts, spacing, layout	The clothes and make-up
JavaScript	The behaviour: what happens when you click, type or scroll	The muscles

Every website, however complex, ends up as HTML, CSS and JavaScript in your browser.

4.3 Kinds of front end

Type	Runs where	Examples	Built with
Web app	In a browser	Gmail, Google Docs	HTML/CSS/JS, often with a framework like React
Mobile app	Installed on a phone	Swiggy, PhonePe	Swift (iPhone), Kotlin (Android), or cross-platform tools like React Native and Flutter
Desktop app	Installed on a computer	Zoom, Slack desktop	Often web technology wrapped as an app
Progressive Web App (PWA)	Browser, but installable like an app	Some news and shopping sites	Web technology

4.4 What belongs in the front end, and what never does

Belongs in the front end:

- layout, design and navigation
- simple checks for convenience ("email address looks incomplete")
- showing data that came from the back end

Must never be only in the front end:

- **secret keys and passwords** for payment or AI services. Anyone can open the browser's developer tools and read front-end code.
- **security decisions** such as "is this user allowed to see this order?". A clever user can bypass front-end checks.
- **price calculations and business rules** that matter. Users can tamper with them.

Golden rule: *Anything in the front end is public. Treat it like a shop window.*

4.5 Other front-end ideas worth knowing

- **Responsive design:** the layout adapts to phone, tablet and laptop screens.
- **Accessibility:** usable by people with disabilities (screen readers, good contrast, keyboard navigation).
- **Client-side vs server-side rendering:** whether the page is assembled in your browser or on the server before it's sent. Server-side rendering usually shows content faster and helps search engines; client-side rendering suits highly interactive apps. Many modern frameworks mix both.

CHAPTER 5 — The Back End

5.1 What it is

The **back end** (also called the **server side**) is the part of the app that runs on servers, out of the user's sight. It is where the real work and the important rules live.

5.2 What the back end does

Job	Plain-English meaning	Food-delivery example
Business logic	The rules of the business	Delivery fee = ₹30 under 3 km, ₹50 above
Authentication	Checking <i>who</i> you are	Verifying your phone number with an OTP
Authorisation	Checking <i>what you're allowed to do</i>	A customer can see only their own orders; a restaurant can see only orders for its kitchen
Validation	Checking data is sensible and safe	Rejecting a quantity of -5 or a delivery address with no pin code
Talking to the database	Saving and fetching information	Saving your order; loading your order history
Talking to other services	Using outside companies' tools	Charging your card through a payment gateway; sending an SMS
Background jobs	Work that happens later or on a schedule	Sending a "rate your meal" message an hour after delivery

Remember the difference: *Authentication* = "Who are you?" (showing your ID card at the gate).
Authorisation = "What are you allowed to do?" (your ID gets you into the library, but not the staff room).

5.3 Back-end languages and frameworks

A **framework** is a ready-made toolkit that saves programmers from writing common things from scratch.

Language	Popular frameworks	Often chosen for
JavaScript / TypeScript (Node.js)	Express, NestJS, Next.js (API routes)	Using one language for front end and back end
Python	Django, FastAPI, Flask	AI, data-heavy apps, fast development
Java	Spring Boot	Large enterprises, banks
C#	ASP.NET	Microsoft-based organisations
PHP	Laravel	Many websites and content systems
Go	Gin, standard library	High-performance services

According to the 2025 Stack Overflow Developer Survey, Node.js and React remained the most-used web technologies, while Python grew strongly on the back of AI and back-end work and the Python framework FastAPI was one of the fastest risers.

5.4 Backend-as-a-Service (BaaS): very common when using LLMs and Gen AI to develop applications

Instead of building a back end from scratch, many apps use a **Backend-as-a-Service** such as **Supabase** or **Firebase**. These give you a ready-made database, user login, file storage and APIs.

- **Upside:** very fast to start. Most AI app builders connect to one automatically.
- **Downside:** the security settings are still *your* responsibility. If you don't configure who can read which data, your whole database can be exposed (see Chapter 12).

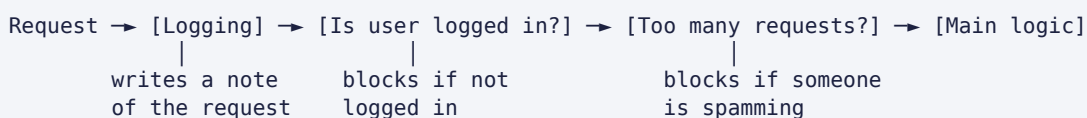
CHAPTER 6 — Middleware

6.1 What it is

Middleware is software that sits *in between* other pieces of software and handles work that is needed on the way through. The word is used in two related ways.

Meaning 1: checkpoints on the path of a request

When a request travels from the front end to the back end's main logic, it often passes through a chain of small checkpoints. Each one does one job and either passes the request on or stops it.



Analogy: airport security. Before you reach the plane (the main logic), you pass ID check, baggage scan and boarding-pass check. Each checkpoint has one job and can stop you.

Common middleware jobs:

- **authentication checks:** is there a valid login token?
- **logging:** recording every request for later investigation
- **rate limiting:** stopping anyone from sending thousands of requests a minute
- **input cleaning:** removing dangerous content from what users typed
- **error handling:** turning crashes into polite error messages
- **CORS rules:** deciding which websites are allowed to call your API

Meaning 2: glue between separate systems

In larger organisations, "middleware" also means software that connects different systems so they can work together:

Type	What it does	Analogy
API gateway	A single front door that receives all API requests and routes them to the right service	Hotel reception directing guests
Message queue (e.g. RabbitMQ, Kafka)	Holds messages until the receiving system is ready	A post box: the sender doesn't wait for the receiver to be home
Integration / connector tools	Moves data between apps (e.g. the CRM and the accounting software)	A translator between two people

Lesson for using LLMs and Gen AI to develop applications: you don't need to build middleware yourself, but you should *ask for it*: "Add rate limiting to the login endpoint", "Log every order placed", "Only logged-in users can reach the /orders API."

CHAPTER 7 — APIs: How the Parts Talk

7.1 What an API is

An **API** (Application Programming Interface) is a defined way for one piece of software to ask another piece of software for something.

Analogy: the restaurant menu and waiter. You don't walk into the kitchen and cook. You choose from a **menu** (the list of things you can ask for), a **waiter** carries your request, and the kitchen sends back your food. An API is the menu plus the waiter. You can only ask for what's on the menu, in the format the menu expects.

7.2 Two kinds of APIs you'll meet

- **Your own APIs:** how your front end talks to your back end ("get my orders", "place an order").
- **Third-party APIs:** services from other companies that your back end uses:
 - payments (Razorpay, Stripe, PayPal)
 - maps and location (Google Maps)
 - messaging (SMS gateways, WhatsApp Business, email services)
 - AI models (Claude, GPT, Gemini)
 - login ("Sign in with Google")

7.3 Anatomy of an API request

Most web APIs follow a style called **REST**. A request has:

Part	Meaning	Example
Endpoint	The address of the specific thing you want	<code>https://api.fooddelivery.com/orders</code>
Method	The action you want to take	GET (fetch), POST (create)
Headers	Extra information, including proof of who you are	An authorisation token or API key
Body	The data you are sending, if any	The dishes and address for a new order

The four main methods map neatly onto the four basic things you can do with data, known as **CRUD**:

Action	CRUD	HTTP method	Example
Create something new	Create	POST	Place a new order
Look something up	Read	GET	View my past orders
Change something	Update	PUT / PATCH	Change the delivery address
Remove something	Delete	DELETE	Cancel a saved card

7.4 What a response looks like

Data usually comes back in **JSON**, a simple, readable format of labels and values:

```
{
  "order_id": 48213,
  "restaurant": "Gujarati Thali House, Navrangpura",
  "items": [
    { "dish": "Paneer Tikka", "quantity": 1, "price": 280 },
    { "dish": "Butter Naan", "quantity": 2, "price": 60 }
  ],
  "total": 400,
  "status": "Out for delivery"
}
```

Every response also includes a **status code**, a three-digit number that says how it went:

Code	Meaning	Plain English
200	OK	Here's what you asked for
201	Created	Your new thing was created
400	Bad request	What you sent doesn't make sense
401	Unauthorised	I don't know who you are; please log in
403	Forbidden	I know who you are, but you're not allowed
404	Not found	That thing doesn't exist
429	Too many requests	Slow down (rate limit reached)
500	Server error	Something broke on our side

Shortcut: 2xx = success, 4xx = the *client* made a mistake, 5xx = the *server* made a mistake.

7.5 API keys and rate limits

- An **API key** is a secret password that proves your app is allowed to use a service. Keys for paid services (payments, AI models) must be kept **in the back end only**, never in front-end code. A leaked key lets strangers spend your money.
- A **rate limit** is the maximum number of requests allowed in a period, e.g. 100 per minute. It protects services from overload and abuse.

7.6 Other API styles, briefly

- **GraphQL:** instead of many fixed endpoints, the client asks for exactly the fields it needs in one request. Useful for complex apps; more to learn.
- **Webhooks:** "Don't call us, we'll call you." Instead of your app repeatedly asking "has the payment gone through yet?", the payment company calls *your* endpoint the moment it has.

- **WebSockets:** a connection that stays open so the server can push live updates, such as a delivery rider's moving dot on a map or chat messages.

CHAPTER 8 — Databases and How to Choose One

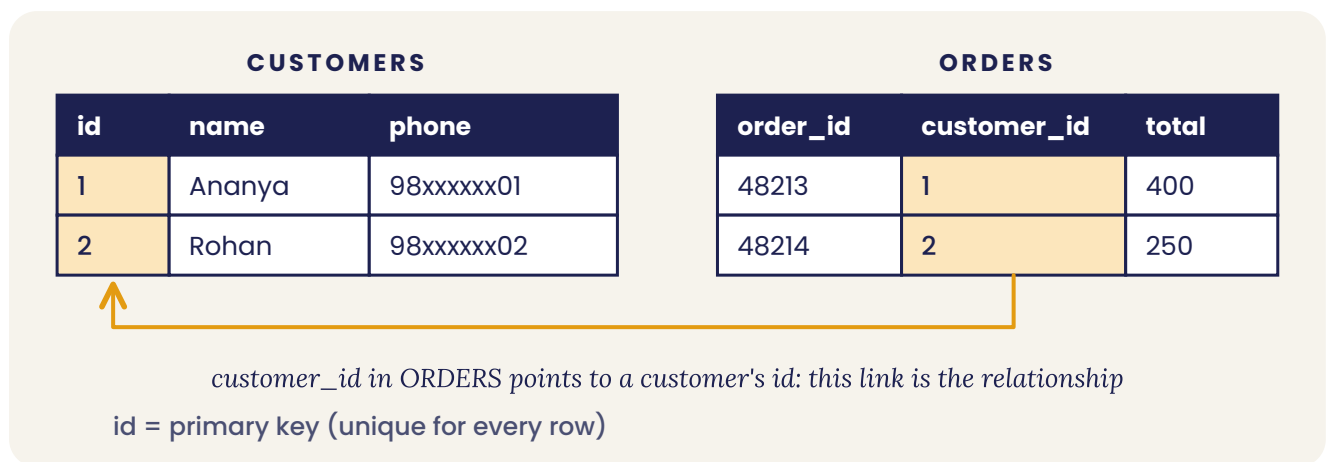
8.1 What a database is

A **database** is an organised, permanent store of information that an app can search and update quickly and safely, even when thousands of people use it at once.

Why not just a spreadsheet? Databases handle many simultaneous users, enforce rules (every order must belong to a real customer), recover from crashes, and find one record among millions in milliseconds.

8.2 Relational (SQL) databases

Data is stored in **tables** of rows and columns, like linked spreadsheets.



- Each row has a unique **primary key** (**id**).
- Tables are linked by **relationships**: **customer_id** in **ORDERS** points to a customer.
- You query them with **SQL** (Structured Query Language).
- Examples: **PostgreSQL**, **MySQL**, **SQLite**, Microsoft SQL Server, Oracle.
- In the 2025 Stack Overflow Developer Survey, PostgreSQL was the most popular database among developers.

Strengths: accuracy, consistency and strong rules. Ideal for money, orders, bookings and inventory: anything where "almost right" is wrong.

8.3 Non-relational (NoSQL) databases

"NoSQL" covers several different designs, each optimised for a particular job:

Type	How data is stored	Good for	Examples
Document	Flexible records, like JSON files	Content where each item can have different fields (product catalogues, user profiles)	MongoDB, Firestore
Key-value	A label and a value, like a locker number and its contents	Very fast lookups, caching, login sessions	Redis, DynamoDB

Type	How data is stored	Good for	Examples
Graph	Points and connections	Relationships: friends of friends, recommendations, fraud rings	Neo4j
Vector	Numerical "meaning fingerprints" of text or images	AI search: "find documents similar in meaning to this question"	Pinecone, pgvector

8.4 Other kinds of storage

- **File / object storage:** for photos, videos and PDFs. Databases store the *link* to the file; the file itself sits in storage such as Amazon S3 or Supabase Storage.
- **Cache:** a small, super-fast temporary store for things requested often (e.g. today's popular restaurants), so the main database isn't asked the same question a thousand times.

8.5 How to choose: a decision guide

If your app...	Choose	Why
handles money, orders, bookings, stock or grades	Relational (PostgreSQL or MySQL)	Accuracy and rules matter most
is a small prototype or runs on one device	SQLite	A single file, no server needed
stores items that vary a lot in shape, or needs very flexible data	Document (MongoDB, Firestore)	No fixed structure required
needs instant lookups of simple values or a cache	Key-value (Redis)	Extremely fast
is about relationships and networks	Graph (Neo4j)	Built for connections
needs AI-powered "search by meaning"	Vector database or PostgreSQL with pgvector	Stores meaning, not just words
stores images, videos or documents	Object storage + a database for the details	Databases are poor at large files

Sensible default for most apps developed using LLMs and Gen AI: a relational database, usually **PostgreSQL** (often provided through Supabase), plus object storage for files. Real apps often combine several of these; that's normal.

8.6 Database words you'll hear

- **Schema:** the design of the tables and columns, the database's blueprint
- **Query:** a question asked of the database
- **Index:** like the index at the back of a book, it makes searches much faster
- **Transaction:** a group of changes that must all succeed or all fail together (money leaves one account *and* arrives in another, or neither happens)

- **Backup:** a saved copy for recovery
- **Row-level security (RLS):** rules inside the database that decide which user can see or change which rows; critical when using Supabase

CHAPTER 9 — Choosing a Front-End Framework

9.1 What a framework is

You can build a front end with plain HTML, CSS and JavaScript. But for anything with many screens and lots of interaction, developers use a **framework**, a ready-made toolkit with a standard way of organising code.

Analogy: building furniture from raw timber vs. from a modular kit. The kit (framework) gives you standard parts and instructions, so you build faster and others can understand your work.

Most frameworks are built around **components**: reusable building blocks such as a "restaurant card", a "cart button" or a "rating stars" widget, used again and again across screens.

9.2 The main options

Framework	What it is	Strengths	Consider when
Plain HTML/CSS/JS	No framework	Simple, nothing to install	One-page tools, landing pages, small prototypes
React	The most widely used UI library (from Meta)	Huge community, endless examples, most AI tools know it very well	Most interactive web apps
Next.js	A framework built on React that also includes server features	Fast pages, good for search engines, front end and simple back end together	Full web apps and public-facing sites
Vue	A gentler alternative to React	Easy to learn, clean	Small-to-medium apps, teams new to frameworks
Angular	A complete, structured framework (from Google)	Strong conventions, popular in enterprises	Large corporate applications
Svelte / SvelteKit	A lightweight, fast framework	Less code, very fast	Performance-focused apps
React Native / Flutter	Build one codebase for both iPhone and Android	Save effort across two platforms	Mobile apps
Swift / Kotlin	Native languages for iPhone / Android	Best performance and access to phone features	Demanding mobile apps

Styling tools such as **Tailwind CSS** are often used alongside any of these to style screens quickly.

9.3 How to choose: questions to ask

1. **Web, mobile or both?** Mobile-first often points to React Native or Flutter.
2. **Does it need to be found on Google?** Public content favours server rendering (Next.js).
3. **How interactive is it?** A simple form-and-table tool may need no framework at all.

4. **Who will maintain it?** Pick what your team or future developers know.

5. **What does your AI tool handle best?** Tools produce better results in popular frameworks because they have seen far more examples.

Sensible default when using LLMs and Gen AI to develop applications: React (often via **Next.js**) with **Tailwind CSS** for web apps. It is the most common choice, AI tools are strongest in it, and help is easy to find. Choose something else only for a clear reason.

CHAPTER 10 — Architecture Styles: One Big Piece or Many Small Ones?

10.1 Monolith

A **monolith** is an application built and deployed as **one single unit**. The front-end serving, the logic and the database access all live in one codebase and go live together.

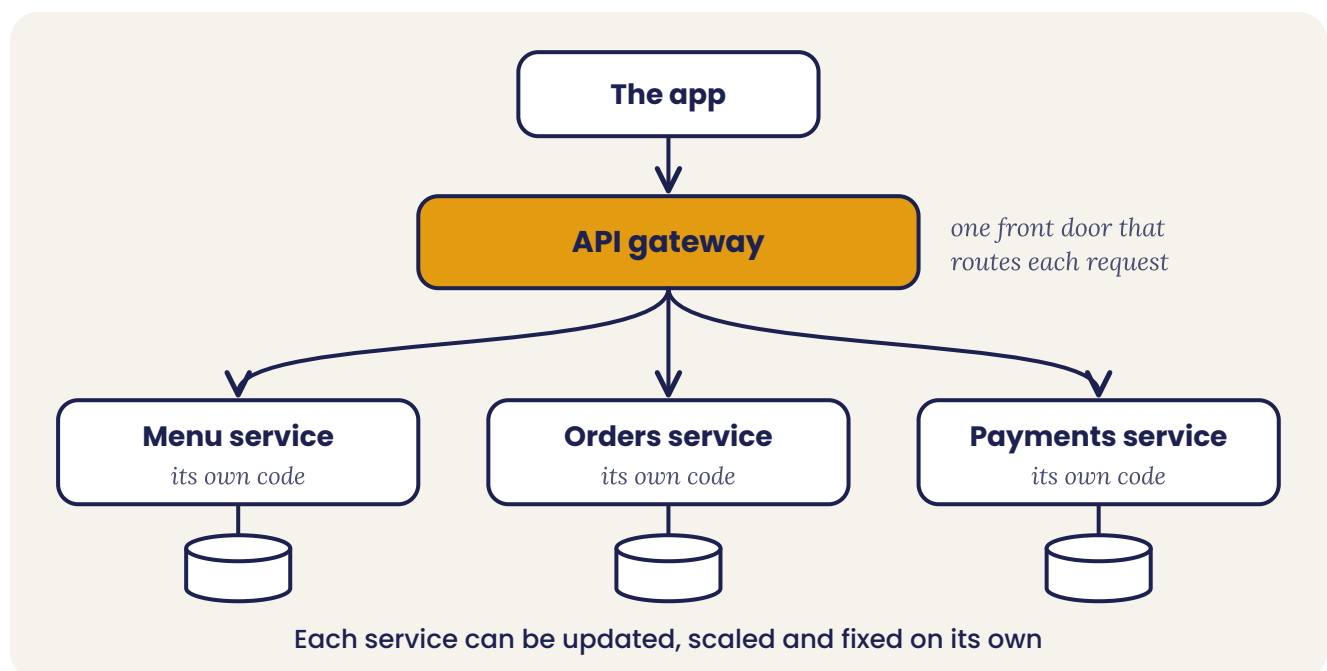
- **Pros:** simple to build, test, deploy and understand; cheaper to run; ideal for small teams
- **Cons:** as it grows very large, changes become riskier, and the whole thing must be redeployed for any change

10.2 Modular monolith

Still one deployable unit, but organised internally into clearly separated **modules** (orders, payments, users), each with clear boundaries. Many experienced engineers see this as the best starting point, and several large companies run successfully on well-organised monoliths; Shopify is a well-known example.

10.3 Microservices

The app is split into many **small, independent services**, each doing one job (menu service, order service, payment service, notification service), each with its own code and often its own database, communicating through APIs.



- **Pros:** teams work independently; each part can be scaled or updated on its own; one failure doesn't have to bring down everything
- **Cons:** far more complex to build, monitor and debug; more expensive; needs mature engineering practices

What the experts say: software thinker Martin Fowler observed in 2015 that nearly all successful microservice systems began as a monolith that grew too big and was later split, and he advised against starting new projects

with microservices. Real companies have moved in both directions. Amazon's Prime Video team moved one video-quality monitoring tool from many separate cloud services back into a single process and cut that tool's infrastructure cost by over 90%. Note that this was one tool, not all of Prime Video. Netflix, meanwhile, moved to microservices to support its global scale.

10.4 Serverless

With **serverless**, you don't manage servers at all. You write small functions, and a cloud provider runs them only when needed, charging per use.

- **Pros:** no server management, scales automatically, cheap at low traffic
- **Cons:** limits on how long each function can run, occasional slow "cold starts", and costs that can rise unexpectedly at high traffic

10.5 Event-driven

Parts of the system react to **events** (things that happened) rather than calling each other directly. "Order placed" is announced once; the kitchen display, the rider-assignment system and the notification system each react to it independently. This is common in large systems and often uses message queues (Chapter 6).

10.6 Which style should you choose when using LLMs and Gen AI to develop applications?

Situation	Recommended style
Prototype, class project, small business tool	Monolith , often a single Next.js app plus a managed database
Growing product with a small team	Modular monolith
Large organisation, many teams, very high scale	Microservices , introduced gradually
Occasional, spiky workloads (e.g. process an uploaded file)	Serverless functions

Rule of thumb: start simple. Split only when you feel real pain, not because it sounds impressive.

CHAPTER 11 — Deployment: From Laptop to the Internet

11.1 What deployment means

Deployment is the process of taking an app that works on one computer and putting it on servers connected to the internet, so real users can reach it at a web address.

Analogy: cooking a dish in your home kitchen (development) vs. serving it in a restaurant to paying customers every day (production). Production needs reliability, hygiene, stock management and a plan for when things go wrong.

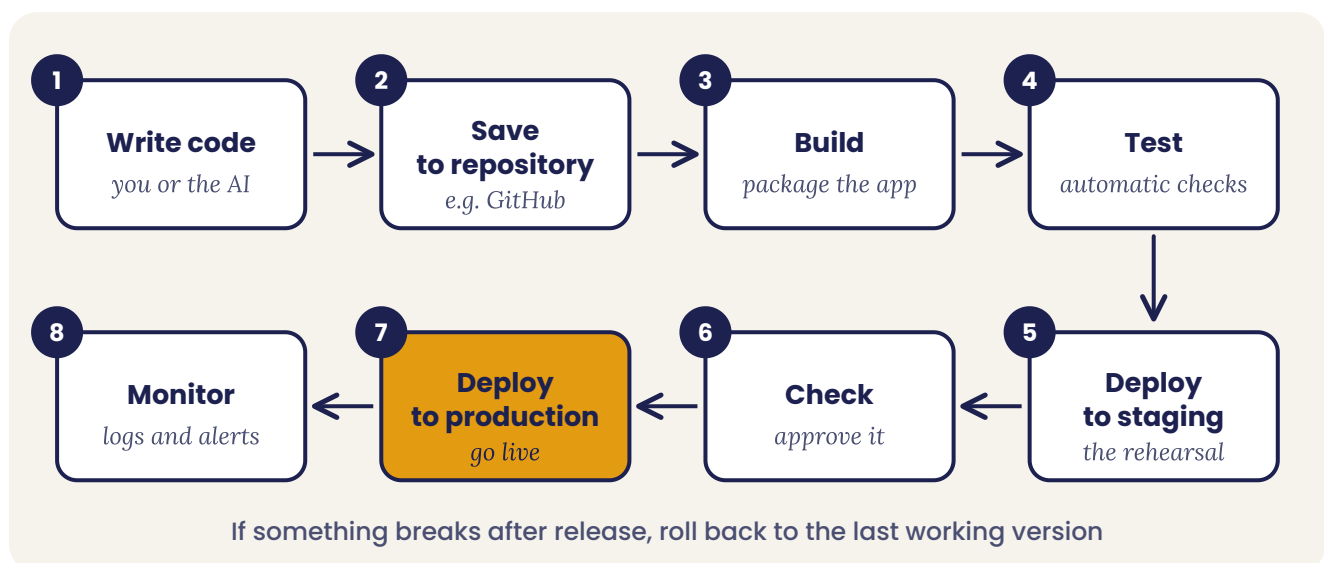
11.2 Environments

Professional teams keep separate copies of an app:

Environment	Purpose	Who uses it
Development (dev)	Building and experimenting	The builder
Staging / test	A realistic rehearsal before going live	Testers, reviewers
Production (prod)	The real, live app	Real users, with real data

Never experiment on production. A well-known 2025 incident involved an AI coding agent deleting a company's live database during a "code freeze". Separating development from production data is the safeguard.

11.3 The deployment pipeline



- **Version control / repository:** a system (usually **Git**, hosted on **GitHub**) that keeps every version of your code, so you can see what changed and go back. For anyone using LLMs and Gen AI to develop applications, this is the "undo button" for the whole project.

- **Build:** converting source code into the optimised files that actually run.
- **CI/CD (Continuous Integration / Continuous Deployment):** automation that tests and deploys your code every time you save a change to the repository.
- **Rollback:** returning quickly to the previous working version if a release breaks something.

11.4 Where to host: the main options

Hosting type	What it is	Good for	Examples
Static hosting + CDN	Serves ready-made files from computers all around the world (a Content Delivery Network)	Websites, simple front ends, landing pages	Netlify, Vercel, GitHub Pages, Cloudflare Pages
Serverless platforms	Run your front end plus small back-end functions on demand	Next.js and similar web apps	Vercel, Netlify
Platform-as-a-Service (PaaS) / containers	Run a back end that stays switched on permanently	APIs, background jobs, live connections, scheduled tasks	Render, Railway, Fly.io
Managed database	A database someone else runs, backs up and updates	Almost every app	Supabase, Neon, MongoDB Atlas
Big cloud providers	Raw computing building blocks with full control	Large or specialised systems	AWS, Google Cloud, Microsoft Azure

Vercel and Netlify are well suited to front ends and serverless functions, but not to back ends that must run continuously; Render or Railway fit those better. The big cloud providers are powerful but usually overkill when you are just starting.

A common, simple setup for apps developed using LLMs and Gen AI: front end on Vercel or Netlify, database and login on Supabase or Firebase, and any long-running back end on Render or Railway.

11.5 Going live: the checklist

- ☐ **Domain name:** buy one (e.g. `mytiffinapp.in`) and point it at your hosting through DNS settings
- ☐ **HTTPS:** a security certificate so traffic is encrypted; most hosts add this free and automatically
- ☐ **Environment variables:** secret keys and settings stored in the hosting platform's settings, *not* written into the code
- ☐ **Database security:** access rules switched on (e.g. row-level security in Supabase)
- ☐ **Backups:** automatic database backups turned on and tested
- ☐ **Monitoring and logs:** you get an alert if the app goes down or errors spike
- ☐ **Costs:** spending limits or alerts set, especially for usage-based pricing and AI APIs

CHAPTER 12 — Security, Scalability, Reliability and Cost

12.1 Security basics every builder must know

Principle	Meaning	Example
Never trust the front end	Re-check everything on the back end	Recalculate the order total on the server, even if the app sent one
Keep secrets secret	API keys only in the back end or environment variables	The payment key never appears in browser code
Authenticate and authorise every request	Check who the user is <i>and</i> what they may access	Rohan can't open <code>/orders/48213</code> because it's Ananya's
Validate input	Reject malformed or malicious data	Block scripts typed into a "name" field
Least privilege	Give each part only the access it needs	The reporting page gets read-only database access
Encrypt	HTTPS for traffic; hashed passwords in storage	Passwords are never stored as plain text
Keep dependencies updated	Third-party code libraries get security fixes	Update regularly

Why this matters especially when using LLMs and Gen AI to develop applications: security researchers keep finding the same holes in AI-built apps.

- A vulnerability catalogued as **CVE-2025-48757** found that 170 of 1,645 analysed apps built with the Lovable platform had missing or weak database access rules (row-level security), letting outsiders read or change data.
- Veracode's 2025 research found security flaws in AI-generated code in 45% of its tests.
- The most common problems are secret keys exposed in front-end code, database access rules switched off, and permission checks done only in the browser.

Your defence: know the fundamentals in this guide, explicitly ask your agent for secure design, and ask it to review its own work for these specific problems before you go live.

12.2 Scalability: handling growth

- **Vertical scaling (scale up):** a bigger, more powerful server. Simple, but there's a ceiling.
- **Horizontal scaling (scale out):** more servers sharing the work. No hard ceiling, but more complex.
- **Load balancer:** distributes incoming requests across several servers, like a supermarket supervisor directing shoppers to free checkout counters.
- **Caching:** keeping frequently requested answers ready.
- **CDN:** copies of files on servers around the world, so users download from somewhere nearby.

12.3 Reliability

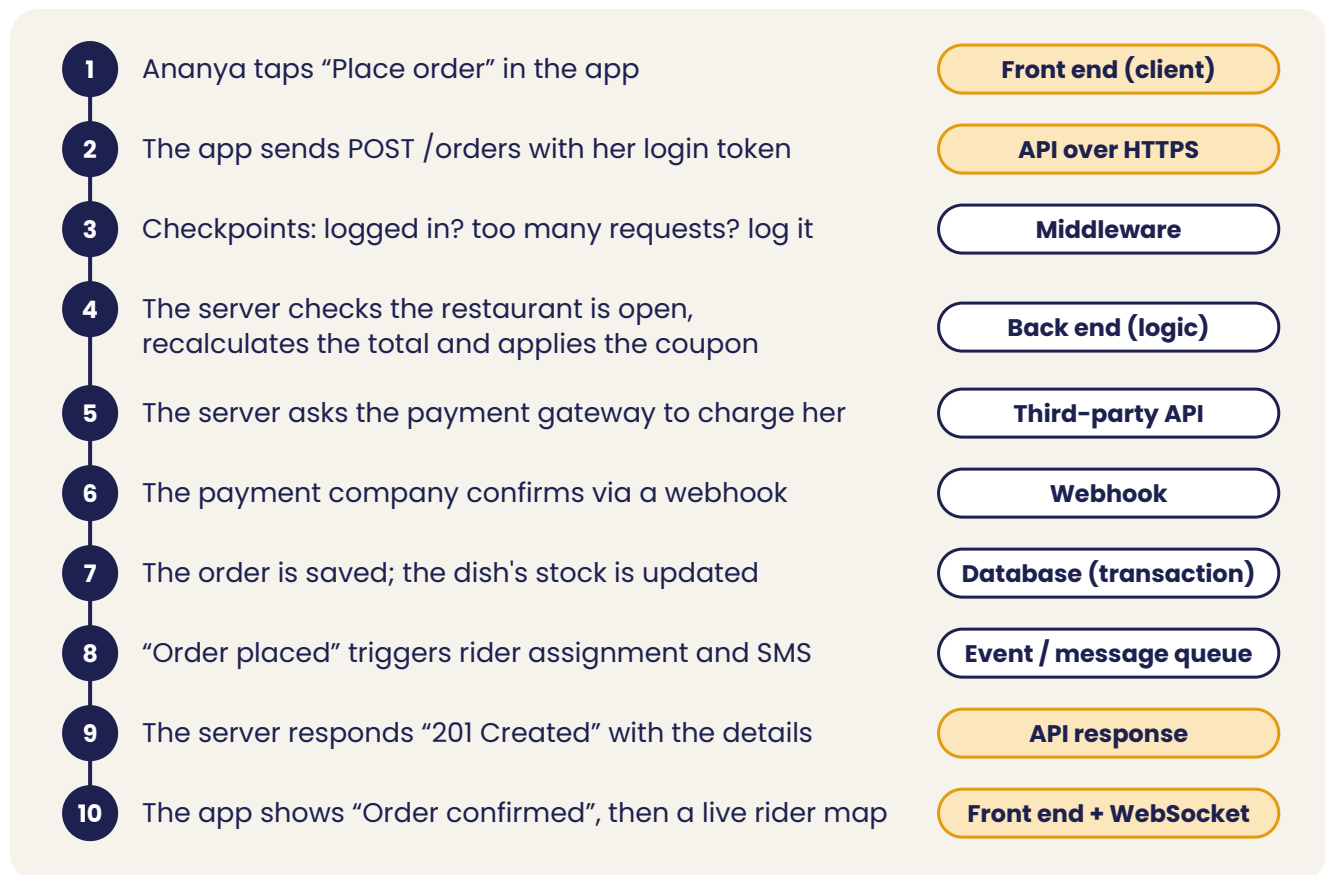
- **Redundancy:** no single point of failure; if one server dies, another takes over
- **Backups:** copies of data you have actually tested restoring
- **Monitoring and alerts:** find problems before users complain
- **Graceful failure:** if the maps service is down, the app still takes orders and shows a friendly message

12.4 Cost

Hosting, databases, bandwidth and especially AI API calls all cost money, and many are **usage-based**: more users or more traffic means a bigger bill. Set budgets and alerts from day one, and prefer free tiers for prototypes.

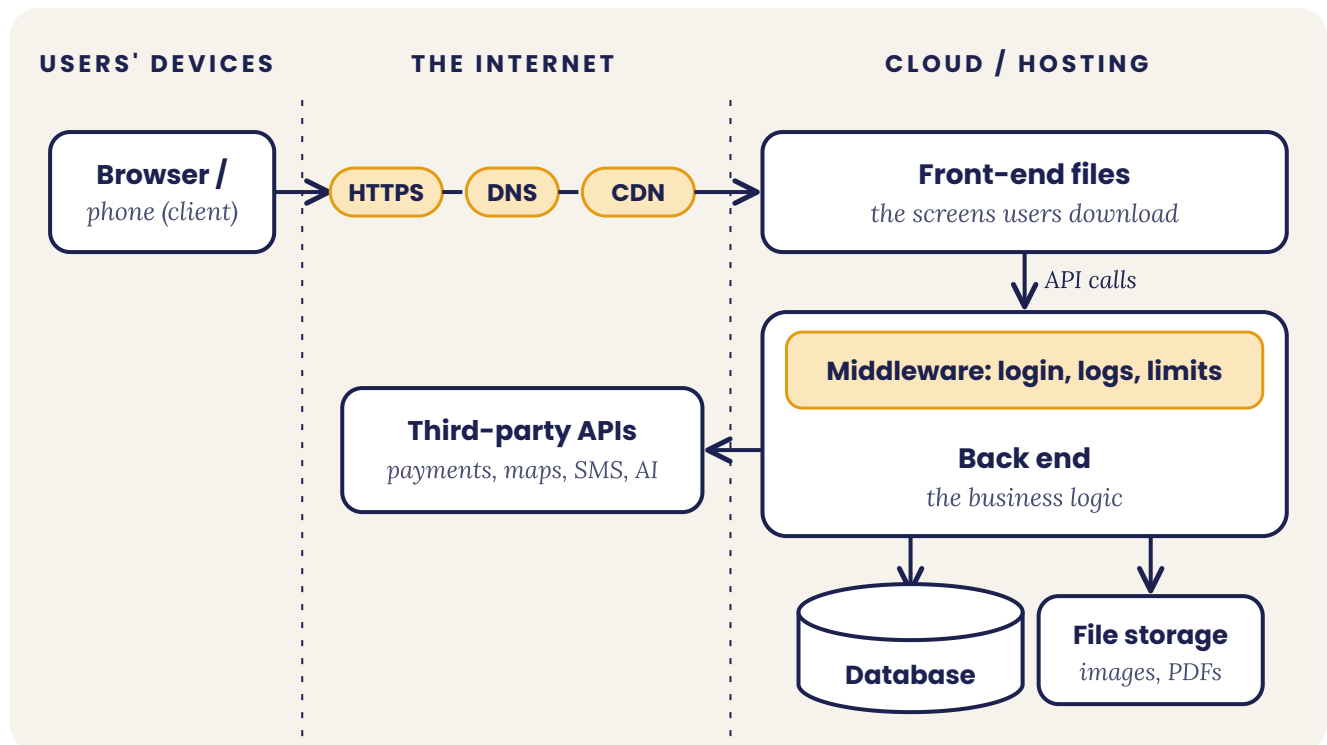
CHAPTER 13 — Putting It All Together

13.1 One food order, end to end



All of this runs on servers that were **deployed** to a hosting provider, behind a **domain name** with **HTTPS**, and **monitored** by the team.

13.2 The architecture diagram of a typical modern app



13.3 How to read the architecture of ANY app: seven questions

Use these to analyse any app, whether it's Instagram, IRCTC or your college's attendance portal:

1. **Who are the users, and what devices do they use?** (the clients)
2. **What do they see and do?** (the front end)
3. **What rules and decisions happen behind the scenes?** (the back end)
4. **What information must be stored, and what kind is it?** (the database choice)
5. **What outside services does it rely on?** (third-party APIs)
6. **What checks happen on every request?** (middleware and security)
7. **Where does it run, and how does it reach users?** (deployment)

GLOSSARY

A. Foundational terms

The words you will hear most often when anyone talks about how an app is built. Learn these first.

Term	Plain-English meaning
Architecture	The high-level plan of an app: its main parts, what each does, how they talk and where they run
Client	The device or program that makes requests (browser, phone app)
Server	A computer or program that responds to requests
Front end	What users see and interact with
Back end	The server-side part where rules and data processing happen
Database	An organised, permanent store of information
API	A defined way for one program to request something from another
API key	A secret password that lets an app use a service
Middleware	Software in between that checks, transforms or connects
Authentication	Checking who someone is
Authorisation	Checking what someone is allowed to do
Framework	A ready-made toolkit with a standard way to build software
HTTP / HTTPS	The language of the web / its secure, encrypted version
URL	A full web address, e.g. https://www.example.com/menu
Domain	A website's name, e.g. <code>example.com</code>
Cloud	Computers rented over the internet from providers such as AWS, Google Cloud or Microsoft Azure
Hosting	Renting space on internet-connected servers so an app can be reached by users
Deployment	Putting an app on internet-connected servers for real users
Production	The live version used by real people

B. Advanced terms

More technical words that appear in later chapters and in conversations with developers or AI coding tools.

Term	Plain-English meaning
API gateway	A single front door that routes API requests to the right service
BaaS	Backend-as-a-Service: ready-made back end (e.g. Supabase, Firebase)
Build	Turning source code into files that can run
Cache	A fast, temporary store for frequently used answers
CDN	A worldwide network of servers that deliver files from near the user
CI/CD	Automation that tests and deploys code after every change
Component	A reusable building block of a user interface
CRUD	Create, Read, Update, Delete: the four basic data actions
DNS	The internet's phone book that turns names into IP addresses
Endpoint	The specific address of one API function
Environment variable	A setting (often a secret) stored outside the code
JSON	A simple text format for sending data
Load balancer	Spreads requests across several servers
Microservices	An app split into many small independent services

B. Advanced terms (continued)

Term	Plain-English meaning
Monolith	An app built and deployed as one single unit
NoSQL	Non-table databases: document, key-value, graph, vector
Rate limit	A cap on how many requests are allowed per period
Relational database	Stores data in linked tables; queried with SQL
Repository (repo)	Where code and its full history are stored (e.g. GitHub)
REST	The most common style of web API
Rollback	Returning to the previous working version
Row-level security (RLS)	Database rules deciding who can see which rows
Serverless	Code runs on demand without you managing servers
SQL	The language used to query relational databases
Staging	A rehearsal copy of the app for testing before going live
Status code	A number telling how a request went (200, 404, 500...)
Webhook	A service calling <i>your</i> app when something happens
WebSocket	A connection kept open for live updates